Experiment 'p_6_2_g_second_largest_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 1
Verification attempts: 1

Problem Specification

Problem name: p_6_2_g_second_largest_seq
Natural language statement: null
Method signature: p_6_2_g_second_largest_seq(inputs: seq<int>) returns (second_largest: int)

Ensures

- `second_largest <= find_largest(inputs)`
- `forall x :: x in inputs ==> x < find_largest(inputs) ==> x <= second_largest`

Requires

- `|inputs| >= 2`

Functional Code Given

```
function find_largest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> x <= ret
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_largest := find_largest(inputs[1..]);
    var ret: int := if inputs[0] >= rest_largest then inputs[0] else rest_largest;
    assert inputs == [inputs[0]] + inputs[1..];
    ret
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_6_2_g_second_largest_seq(inputs: seq<int>) returns (second_largest: int)

The method should respect the following contract:

requires |inputs| >= 2, ensures second_largest <= find_largest(inputs), ensures forall x
    ↪ :: x in inputs ==> x < find_largest(inputs) ==> x <= second_largest

The contract uses the following dafny code:

function find_largest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> x <= ret
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_largest := find_largest(inputs[1..]);
    var ret: int := if inputs[0] >= rest_largest then inputs[0] else rest_largest;
    assert inputs == [inputs[0]] + inputs[1..];
    ret
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnNz2RdfX6wjy2fPFsgtxnckoWpai
Model: gpt-5.5-2026-04-23
Created at: 1780662636
Finish reason: stop
Usage: 610 tokens in, and 3626 tokens out

```
{
  var i := 1;
  second_largest := inputs[0];
```

```
while i < |inputs|
  invariant 1 <= i <= |inputs|
  invariant second_largest in inputs[..i]
  invariant forall x :: x in inputs[..i] ==> x <= second_largest
  decreases |inputs| - i
{
  var oldSecond := second_largest;

  if inputs[i] > oldSecond {
    second_largest := inputs[i];
  }

  assert inputs[..i + 1] == inputs[..i] + [inputs[i]];

  assert second_largest in inputs[..i + 1] by {
    if inputs[i] > oldSecond {
      assert second_largest == inputs[i];
      assert second_largest in [inputs[i]];
      assert second_largest in inputs[..i] + [inputs[i]];
    } else {
      assert second_largest == oldSecond;
      assert second_largest in inputs[..i];
      assert second_largest in inputs[..i] + [inputs[i]];
    }
  }

  assert forall x :: x in inputs[..i + 1] ==> x <= second_largest by {
    forall x | x in inputs[..i + 1]
      ensures x <= second_largest
    {
      assert x in inputs[..i] + [inputs[i]];
      if x in inputs[..i] {
        assert x <= oldSecond;
        if inputs[i] > oldSecond {
          assert oldSecond < second_largest;
        } else {
          assert second_largest == oldSecond;
        }
      } else {
        assert x in [inputs[i]];
        assert x == inputs[i];
        if inputs[i] > oldSecond {
          assert second_largest == inputs[i];
        } else {
          assert second_largest == oldSecond;
          assert inputs[i] <= second_largest;
        }
      }
    }
  }
```

```
    i := i + 1;
  }

  assert inputs[..i] == inputs;
  assert second_largest in inputs;
  assert forall x :: x in inputs ==> x <= second_largest;
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_6_2_g_second_largest_seq/program.dfy(48,11): Warning: Could not find a
    ↪ trigger for this quantifier. Without a trigger, the quantifier may cause brittle
    ↪ verification. To silence this warning, add an explicit trigger using the {:trigger
    ↪ } attribute. For more information, see the section on quantifier instantiation
    ↪ rules in the reference manual.
   |
48 | assert forall x :: x in inputs[..i + 1] ==> x <= second_largest by {
   | ^^^^^^

problem_data/p_6_2_g_second_largest_seq/program.dfy(49,6): Warning: Could not find a
    ↪ trigger for this quantifier. Without a trigger, the quantifier may cause brittle
    ↪ verification. To silence this warning, add an explicit trigger using the {:trigger
    ↪ } attribute. For more information, see the section on quantifier instantiation
    ↪ rules in the reference manual.
   |
49 | forall x | x in inputs[..i + 1]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Compilation failed because warnings were found and --allow-warnings is false

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnO0WsGDimYKXiHQGmhgHNXpMAMx9
Model: gpt-5.5-2026-04-23
Created at: 1780662728
Finish reason: stop
Usage: 1442 tokens in, and 1590 tokens out

```
{
  var i := 1;
  second_largest := inputs[0];

  while i < |inputs|
```

```
  invariant 1 <= i <= |inputs|
  invariant second_largest in inputs[..i]
  invariant forall x {:trigger x in inputs[..i]} :: x in inputs[..i] ==> x <=
      ↪ second_largest
  decreases |inputs| - i
{
  var oldSecond := second_largest;

  if inputs[i] > oldSecond {
    second_largest := inputs[i];
  }

  assert inputs[..i + 1] == inputs[..i] + [inputs[i]];

  assert second_largest in inputs[..i + 1] by {
    if inputs[i] > oldSecond {
      assert second_largest == inputs[i];
      assert second_largest in [inputs[i]];
      assert second_largest in inputs[..i] + [inputs[i]];
    } else {
      assert second_largest == oldSecond;
      assert second_largest in inputs[..i];
      assert second_largest in inputs[..i] + [inputs[i]];
    }
  }

  assert forall x {:trigger x in inputs[..i + 1]} :: x in inputs[..i + 1] ==> x <=
      ↪ second_largest by {
    forall x {:trigger x in inputs[..i + 1]} | x in inputs[..i + 1]
      ensures x <= second_largest
    {
      assert x in inputs[..i] + [inputs[i]];
      if x in inputs[..i] {
        assert x <= oldSecond;
        if inputs[i] > oldSecond {
          assert oldSecond < second_largest;
        } else {
          assert second_largest == oldSecond;
        }
      } else {
        assert x in [inputs[i]];
        assert x == inputs[i];
        if inputs[i] > oldSecond {
          assert second_largest == inputs[i];
        } else {
          assert second_largest == oldSecond;
          assert inputs[i] <= second_largest;
        }
      }
    }
  }
```

```
    i := i + 1;
  }

  assert inputs[..i] == inputs;
  assert second_largest in inputs;
  assert forall x {:trigger x in inputs} :: x in inputs ==> x <= second_largest;
}
```

Final Program

```
function find_largest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> x <= ret
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_largest := find_largest(inputs[1..]);
    var ret: int := if inputs[0] >= rest_largest then inputs[0] else rest_largest;
    assert inputs == [inputs[0]] + inputs[1..];
    ret
}

method p_6_2_g_second_largest_seq(inputs: seq<int>) returns (second_largest: int)
       requires |inputs| >= 2
       ensures second_largest <= find_largest(inputs)
       ensures forall x :: x in inputs ==> x < find_largest(inputs) ==> x <=
           ↪ second_largest
{
  var i := 1;
  second_largest := inputs[0];

  while i < |inputs|
    invariant 1 <= i <= |inputs|
    invariant second_largest in inputs[..i]
    invariant forall x {:trigger x in inputs[..i]} :: x in inputs[..i] ==> x <=
        ↪ second_largest
    decreases |inputs| - i
  {
    var oldSecond := second_largest;

    if inputs[i] > oldSecond {
      second_largest := inputs[i];
    }

    assert inputs[..i + 1] == inputs[..i] + [inputs[i]];

    assert second_largest in inputs[..i + 1] by {
      if inputs[i] > oldSecond {
```

```
      assert second_largest == inputs[i];
      assert second_largest in [inputs[i]];
      assert second_largest in inputs[..i] + [inputs[i]];
    } else {
      assert second_largest == oldSecond;
      assert second_largest in inputs[..i];
      assert second_largest in inputs[..i] + [inputs[i]];
    }
  }

  assert forall x {:trigger x in inputs[..i + 1]} :: x in inputs[..i + 1] ==> x <=
      ↪ second_largest by {
    forall x {:trigger x in inputs[..i + 1]} | x in inputs[..i + 1]
      ensures x <= second_largest
    {
      assert x in inputs[..i] + [inputs[i]];
      if x in inputs[..i] {
        assert x <= oldSecond;
        if inputs[i] > oldSecond {
          assert oldSecond < second_largest;
        } else {
          assert second_largest == oldSecond;
        }
      } else {
        assert x in [inputs[i]];
        assert x == inputs[i];
        if inputs[i] > oldSecond {
          assert second_largest == inputs[i];
        } else {
          assert second_largest == oldSecond;
          assert inputs[i] <= second_largest;
        }
      }
    }
  }

  i := i + 1;
 }

 assert inputs[..i] == inputs;
 assert second_largest in inputs;
 assert forall x {:trigger x in inputs} :: x in inputs ==> x <= second_largest;
}
```

Total Token Usage

Input tokens: 2052
Output tokens: 5216
Reasoning tokens: 4098
Sum of 'total tokens': 7268

Experiment Timings

Overall Experiment started at 1780662635936, ended at 1780662775524, lasting 139588ms (139.59 seconds)
Iteration #1 started at 1780662635940, ended at 1780662724676, lasting 88736ms (88.74 seconds)
Iteration #2 started at 1780662724676, ended at 1780662775523, lasting 50847ms (50.85 seconds)